

Linking head ...

```
$ ./head
*** Failed! Falsifiable (after 3 tests):
[0]
```

The QuickCheck library generated test cases for different `[Int]` types and it returned a failure after the third test, for which the input was `[0]`. Clearly, the `headExists` `[0]` computation will return `'True'` and not `'False'`.

The way we defined the property is incorrect. We know that if the list is empty, then its length is zero. We can write a helper function `lengthZero` for the above, as follows:

```
lengthZero :: [Int] -> Bool
lengthZero list
  | length list == 0 = True
  | otherwise = False
```

We can then use this function to assert that for any `Integer` list, if `headExists` returns `'False'` then the `lengthZero` function must return `'True'`. The complete code is shown below:

```
import Data.List
import Test.QuickCheck

headExists :: [Int] -> Bool
headExists list
  | null list      = False
  | otherwise = True

lengthZero :: [Int] -> Bool
lengthZero list
  | length list == 0 = True
  | otherwise = False

prop_headExists :: [Int] -> Bool
prop_headExists list = headExists list == not (lengthZero list)

main :: IO ()
main = quickCheck prop_headExists
```

Executing the code produces the required output:

```
$ ghc --make head.hs
[1 of 1] Compiling Main                (head.hs, head.o )
Linking head ...

$ ./head
+++ OK, passed 100 tests.
```

We can also re-write the above code based on conditional properties. The property that the `headExists` function will return `'True'` only for non-empty lists can be defined as a constraint. The notation syntax is `condition ==> property`. In our example, if the condition that the list is non-empty is `'True'`, then the property that the `headExists` function for the list must return is `'True'`. Also, when the list is empty, the `headExists` function must return `'False'`. These two conditions can be written as follows:

```
import Data.List
import Test.QuickCheck

headExists :: [Int] -> Bool
headExists list
  | null list      = False
  | otherwise = True

prop_headExists :: [Int] -> Property
prop_headExists list = length list > 0 ==> headExists list == True

prop_emptyList :: [Int] -> Property
prop_emptyList list = length list == 0 ==> headExists list == False

main :: IO ()
main = do
  quickCheck prop_headExists
  quickCheck prop_emptyList
```

Testing the code produces the following output:

```
$ ghc --make cond.hs
[1 of 1] Compiling Main                (cond.hs, cond.o )
Linking cond ...

$ ./cond
+++ OK, passed 100 tests.
*** Gave up! Passed only 38 tests.
```

These tests can be integrated with `Hspec` or `HUnit` for a more verbose output.

Cabal

Cabal is a software tool that is used to describe a Haskell application, list its dependencies, and provide a manifestation to distribute the source and binaries. It is not to be confused with a distribution package manager like RPM or the Debian package management system. You can install Cabal using your distribution package manager. On Fedora, for example, you can use the following command: